

JAVA BASICS

1. What is a Program / Code?

A program (or code) is a set of instructions written to make a computer perform a specific task or solve a problem.

Simple real-life idea

Task: Make tea

- > Take a vessel
- > Boil water
- > Add tea / milk / sugar
- > Wait for a few minutes
- > Serve

Idea -> Programming language -> Machine-executable instructions

A programming language is a language used by developers to write programs. Examples: C, C++, Java, Python and C#.

2. Java History

Short history — exactly 8 lines

1. 1991: Sun Microsystems began the Green Project to explore software for consumer electronic devices.
2. James Gosling led the work, with contributions from Patrick Naughton and other engineers.
3. The language was first called Oak and was designed for embedded / consumer-electronic applications.
4. 1992: an early JVM design evolved to support the Oak programming language.
5. Oak was later retargeted toward the Internet, revised substantially, and renamed Java.
6. 1995: Sun publicly introduced Java during the SunWorld conference.
7. 1996: the first official JDK 1.0 release followed, establishing the early Java platform.
8. Java's key idea: compile to bytecode, then run that bytecode on a suitable JVM.

Memory hook

Oak -> Internet -> Java -> Bytecode -> JVM -> Portable execution

History verified against Oracle's Java history and language/JVM specifications.

3. What is an Application?

An application is software made from programs / code that performs a useful task for a user or another system.

Common application categories

- > Standalone / desktop: mainly runs on a local computer after installation.
- > Web application: accessed through a browser and communicates with server-side systems.
- > Mobile / client-server application: a mobile client may communicate with backend services.

Important: an application is the software product; Java is the programming language and platform used to build many kinds of applications.

4. Source File -> Bytecode

Java source code is normally saved with the java extension.

```
Hello.java
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello Java");
    }
}
```



javac

Java compiler

Compilation result

The javac compiler checks the source and, when compilation succeeds, produces a .class file containing bytecode.

.java -> javac -> .class (bytecode)

Bytecode is not native machine code. It is designed to be executed by a JVM implementation.

5. JDK — Java Development Kit

Definition: the JDK is the development kit used to develop, compile, test, debug and run Java applications.

Think: JDK = developer's toolbox

javac	compiles java source into .class bytecode
java	launches a Java application through the runtime
javadoc	generates API documentation
jdb	debugging tool
other tools	packaging, monitoring and development utilities

If you want to write and compile Java programs, you normally install a JDK.

Memory

JDK = development tools + runtime components needed to run Java programs.

6. JRE — Java Runtime Environment

Definition: the JRE is the runtime environment required to run Java applications. Conceptually: JVM + Java runtime libraries + supporting runtime files.



+ runtime libraries / supporting files

Key purpose: execution, not development.

Memory trick

JRE = Run

JDK = Develop + Run

JVM = Execute bytecode

Terminology note: modern Java distributions are packaged differently across vendors and releases, but this JDK/JRE/JVM model is a useful foundation for learning.

7. JVM — Java Virtual Machine

Definition: the JVM is the virtual execution engine that loads Java bytecode, verifies it, manages runtime memory and executes the program.

What the JVM does

- > 1. Class loading — loads required .class files.
- > 2. Bytecode verification — checks structural and safety constraints.
- > 3. Runtime memory management — provides areas used by executing code.
- > 4. Execution — interprets bytecode and/or compiles hot code with JIT.
- > 5. Garbage collection — automatically reclaims eligible heap memory.

JVM is not the Java language itself. It is the runtime engine that implements the JVM specification.

Why Java is portable

The same bytecode can run on different operating systems when a compatible JVM exists for each platform.

8. How Java Works – Complete Flow



JIT = Just-In-Time compiler. During execution, the JVM can identify frequently executed (“hot”) code and compile it into native machine instructions for faster repeated execution.

One-line interview answer

Java source is compiled into bytecode; the JVM loads and executes that bytecode, using interpretation and JIT compilation where appropriate.

9. Compile Time vs Runtime

Compile Time

Question: Can the source code be translated successfully?

Typical issues: • syntax errors
• type errors • invalid declarations

Runtime

Question: What happens while the compiled program is executing?

Examples: • division by zero • null access • file / network failures

Successful compilation does not guarantee successful execution.

```
int a = 10;  
int b = 0;  
System.out.println(a / b);
```

This can compile successfully, but execution throws an exception because the division uses zero.

10. JDK vs JRE vs JVM

Component	Main purpose	Contains / does	Think of it as
JDK	Development	Compiler + tools + runtime	Developer toolbox
JRE	Running	JVM + runtime libraries	Run environment
JVM	Execution	Loads, verifies, executes bytecode	Virtual machine

Best way to remember

JDK -> I am building Java software.

JRE -> I need the environment to run Java software.

JVM -> I am the engine executing Java bytecode.

11. Quick Revision Sheet

Program	A set of instructions that performs a task.
Programming language	A language used to write programs.
Java source file	Usually saved with java extension.
Compiler	javac translates Java source into bytecode.
Bytecode	Instructions stored in .class files for the JVM.
JVM	Loads, verifies and executes bytecode.
JRE	Runtime environment model: JVM + runtime libraries.
JDK	Development kit containing tools used to build Java applications.
JIT	Compiles frequently executed code into native code during runtime.

Suggested JAVA-GIT structure

```
JAVA-GIT/  
01-Java-Basics/  
|-- Java-History-JDK-JRE-JVM.pdf  
|-- programs/  
`-- README.md
```